Module 4 — Descriptive Analytics & Visualization

You know how to get data from a database, clean it, and move it through a pipeline. But having data is not the same as understanding data. A pipeline that extracts 50,000 rows and loads them into a table tells you nothing about what those rows mean — whether the numbers are reasonable, which variables are related, whether that spike in revenue last month was signal or noise.

Descriptive analytics is the practice of examining data systematically before making any decisions about it. You look at distributions, measure relationships, test whether observed patterns are statistically real, and communicate your findings with visualizations that a non-technical audience can act on. This module takes you from "I can get data" to "I can understand data and explain what it means."

What success looks like: By the end of this module, you can conduct a systematic exploratory data analysis on a tabular dataset, apply hypothesis testing to validate observed patterns, design a Key Performance Indicator (KPI) framework for a business domain, create publication-quality visualizations with Matplotlib, Seaborn, and Plotly, and produce a descriptive analytics report that integrates findings into a coherent narrative supporting data-driven decisions.

In This Reading

1. What Is EDA and Why Does It Matter?
2. Distributions and Summary Statistics
3. Missing Data and Outliers
4. Correlation and Relationships
5. Hypothesis Testing
6. KPI Design and Cohort Analysis
7. Visualization as Communication

- Troubleshooting
- Key Terms
- Summary
- What's Next

---

1. What Is EDA and Why Does It Matter?

The EDA Mindset

Exploratory Data Analysis (EDA) is the systematic examination of a dataset's structure, distributions, relationships, and anomalies. It happens *before* you model, report, or decide anything. EDA is hypothesis-generating — you are looking for patterns, not confirming them. Confirmation comes later, with statistical tests.

The distinction matters. If you skip EDA and go straight to modeling, you build on assumptions you have not checked. If the data is skewed, your mean is misleading. If two variables are correlated because of a confounding factor, your model learns the wrong relationship. If 20% of your values are missing in a non-random pattern, imputing with the mean introduces bias. EDA catches these problems before they compound.

The EDA Workflow

Every EDA follows a predictable sequence:

1. **Shape and types:** How many rows and columns? What are the data types? Are there columns that look numeric but are actually categorical (e.g., `customer_tier` with values 1, 2, 3)?
2. **Missing values:** How much is missing? Which columns? Is the missingness random or systematic?
3. **Distributions:** What does each variable look like? Symmetric? Skewed? Bimodal? Are there obvious outliers?
4. **Relationships:** Which variables are correlated? Are there grouping effects (e.g., GPA differs by department)?
5. **Outliers:** Are extreme values data errors or genuine extremes? Do they distort the statistics?
6. **Findings summary:** What patterns did you find? What should be investigated further? What should the next analyst know?

Initial Inspection Tools

Start every EDA with these Pandas methods:

```
import pandas as pd

df = pd.read_csv("data/dataset.csv")

df.shape          # (rows, columns)
df.dtypes         # data type of each column
df.describe()     # count, mean, std, min, 25%, 50%, 75%, max for numeric columns
df.info()         # non-null counts and memory usage
df.head()         # first 5 rows — sanity check
```

`df.describe()` gives you a fast overview: if `min` is negative for a column that should only be positive, something is wrong. If `mean` and `50%` (median) are far apart, the distribution is skewed. If `count` differs across columns, you have missing values.

Why EDA Prevents Modeling Errors

Without EDA, three categories of error go undetected:

- **Garbage-in-garbage-out:** If the data has systematic errors (sentinel values like 999 masquerading as real data, duplicated rows, wrong units), any analysis on it is wrong.
- **Violated assumptions:** Many statistical methods assume normal distributions, no multicollinearity, or homoscedasticity. EDA reveals whether these assumptions hold.
- **Data leakage:** If a feature contains information about the target that would not be available at prediction time, EDA's correlation analysis can flag the suspiciously perfect relationship.

Connecting to Module 3

In Module 3, you extracted data from PostgreSQL, transformed it with Pandas, and loaded results into tables and CSVs. You know how to get data — joins, aggregations, CTEs. Module 4 is about what you do after the data lands in a DataFrame. The Extract, Transform, Load (ETL) pipeline you built is the input; descriptive analytics is the analysis layer.

---

2. Distributions and Summary Statistics

Measures of Center

A **measure of center** answers "what is a typical value?"

- **Mean** (arithmetic average): `df['col'].mean()`. Sensitive to outliers — one billionaire in a room of teachers makes the mean salary misleading.
- **Median** (middle value when sorted): `df['col'].median()`. Robust to outliers. For skewed data, the median is almost always the better choice.
- **Mode** (most frequent value): `df['col'].mode()`. Useful for categorical data (most common category) and multimodal distributions.

Decision rule: If the distribution is symmetric, mean and median are close — use either. If the distribution is skewed, use the median. Report both when they diverge significantly.

Measures of Spread

A **measure of spread** answers "how much do values vary?"

- **Standard deviation** (`df['col'].std()`): Average distance from the mean. Measured in the same units as the data. Useful when the distribution is approximately normal.
- **Interquartile Range (IQR)**: Q3 minus Q1 — the range of the middle 50% of values. Robust to outliers.
- **Range** (`max - min`): Simplest but most sensitive to outliers. Rarely informative alone.

Distribution Shapes

Understanding the *shape* of a distribution tells you more than any single number:

- **Normal (bell curve):** Symmetric, mean ≈ median. Many natural phenomena approximate this shape.
- **Right-skewed (positive skew):** Long tail on the right. Mean > median. Common for income, prices, durations — anything with a natural floor (zero) and no ceiling.
- **Left-skewed (negative skew):** Long tail on the left. Mean < median. Less common — examples include GPA distributions where most students cluster high.
- **Bimodal:** Two distinct peaks. Suggests two subpopulations in the data (e.g., heights of mixed adult/child sample).
- **Uniform:** Roughly equal frequency across the range. Rare in real data; might indicate generated or bucketed data.

Skewness and kurtosis are numeric measures of shape:

- **Skewness** (`df['col'].skew()`): 0 is symmetric, positive means right-skewed, negative means left-skewed. Values beyond ±1 indicate substantial skew.
- **Kurtosis** (`df['col'].kurtosis()`): Measures tail heaviness relative to a normal distribution. High kurtosis means more extreme outliers. In practice, check it when you plan to use methods that assume normality.

Visualizing Distributions

Histograms divide a continuous variable into bins and count observations per bin. They are the workhorse of distribution visualization.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Matplotlib histogram
fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(df['gpa'], bins=20, edgecolor='black', alpha=0.7)
ax.set_xlabel('GPA')
ax.set_ylabel('Count')
ax.set_title('Distribution of Student GPA')
plt.tight_layout()
fig.savefig('output/gpa_histogram.png', dpi=150)
plt.close()
```

```
# Seaborn histogram with Kernel Density Estimation (KDE) overlay
fig, ax = plt.subplots(figsize=(8, 5))
sns.histplot(df['gpa'], kde=True, bins=20, ax=ax)
ax.set_title('GPA Distribution with Density Estimate')
plt.tight_layout()
fig.savefig('output/gpa_histplot.png', dpi=150)
plt.close()
```

Bin width matters. Too few bins hide patterns. Too many bins create noise. Start with 20–30 bins and adjust. Seaborn's `histplot` uses an automatic algorithm that works well for most data sets.

Box plots show the five-number summary (min, Q1, median, Q3, max) plus outliers:

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.boxplot(x='department', y='gpa', data=df, ax=ax)
ax.set_title('GPA by Department')
plt.tight_layout()
fig.savefig('output/gpa_by_department.png', dpi=150)
plt.close()
```

When to use a box plot instead of a histogram: When you are comparing distributions across groups (e.g., GPA by department). Histograms become cluttered with more than 2–3 overlapping distributions; box plots remain clean.

Density plots (`sns.kdeplot()`) produce a smooth, continuous estimate of the distribution — a Kernel Density Estimation (KDE). Useful for overlaying multiple distributions on the same axes:

```
fig, ax = plt.subplots(figsize=(8, 5))
for dept in df['department'].unique():
    subset = df[df['department'] == dept]
    sns.kdeplot(subset['gpa'], label=dept, ax=ax)
ax.set_title('GPA Distribution by Department')
ax.legend()
plt.tight_layout()
fig.savefig('output/gpa_density_by_dept.png', dpi=150)
plt.close()
```

---

3. Missing Data and Outliers

Detecting Missing Values

Missing data is inevitable. The question is not whether your dataset has it, but *how much* and *why*.

```
# Count missing values per column
df.isnull().sum()

# Percentage missing per column
(df.isnull().sum() / len(df) * 100).round(2)

# Quick overview
df.info()    # shows non-null counts per column
```

Not all missing data looks like `NaN`. Watch for **sentinel values** — magic numbers or strings that represent "unknown":

- Numeric: 999, -1, 0 (when 0 is not a valid value)
- String: "N/A", "Unknown", "NA", empty string ""
- Date: 1900-01-01, 9999-12-31

Replace sentinels with proper `NaN` before analysis: `df.replace(999, pd.NA, inplace=True)`.

Also watch for **implicit gaps** — data that should exist but does not. If you have daily sales data but certain dates are simply absent (not NaN, just missing rows), that is a data quality problem your null counts will not catch. Check for completeness: do you have all expected dates, all expected customer IDs, all expected categories?

Missing Data Patterns

Understanding *why* data is missing determines how to handle it:

- **MCAR (Missing Completely At Random):** The missingness has no relationship to any variable. Example: a sensor randomly fails to record 2% of readings. Safe to drop — the remaining data is an unbiased sample.
- **MAR (Missing At Random):** The missingness is related to an observed variable but not to the missing value itself. Example: older survey respondents skip the income question more often. You can model the missingness using the observed variables and impute.
- **MNAR (Missing Not At Random):** The missingness is related to the value that is missing. Example: high earners decline to report income — the very people whose income is missing have systematically different incomes. This is the hardest pattern — no simple fix.

In practice, you cannot definitively prove which pattern you have. But you can check: are the rows with missing values different in other columns from the rows without? If `commute_minutes` is missing more often for students with low attendance, the missingness may not be random.

Visualizing Missing Data

A heatmap of the null matrix reveals patterns at a glance:

```
fig, ax = plt.subplots(figsize=(10, 6))
sns.heatmap(df.isnull(), cbar=False, yticklabels=False, ax=ax)
ax.set_title('Missing Data Pattern')
plt.tight_layout()
fig.savefig('output/missing_data_heatmap.png', dpi=150)
plt.close()
```

If columns have missing values in the same rows, they may be missing for the same reason (MAR or MNAR with a shared cause).

Handling Missing Data

Strategy	When to Use	Tradeoff
Drop rows (`df.dropna()`)	<5% missing, MCAR	Reduces sample size; safe if data is abundant
Drop columns (`df.drop(columns=[...])`)	>50% missing, column is not critical	Loses the variable entirely
Impute with mean	Numeric, roughly symmetric distribution	Reduces variance; biased if data is skewed
Impute with median	Numeric, skewed distribution	More robust than mean for skewed data
Impute with mode	Categorical	Safe for dominant categories; risky if distribution is flat
Impute with group statistics	Variable differs by group	E.g., impute commute_minutes with median per department
Flag as indicator	Missingness itself might be informative	Add a `col_missing` boolean column before imputing

Key rule: imputing with the mean when the data is heavily skewed shifts the distribution toward the center and inflates the central tendency. Use the median for skewed data.

Document your choices. Every imputation decision should be recorded — what was missing, what method you chose, and why. Future analysts (including future you) need to know.

Outlier Detection

An **outlier** is a data point far outside the expected range. But "far" needs a definition.

IQR method:

```
Q1 = df['col'].quantile(0.25)
Q3 = df['col'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df['col'] < lower_bound) | (df['col'] > upper_bound)]
```

This is the same method box plots use to determine which points are plotted individually beyond the whiskers.

Z-score method:

```
from scipy import stats

df['z'] = stats.zscore(df['col'], nan_policy='omit')
outliers = df[df['z'].abs() > 3]
```

A z-score above 3 means the value is more than 3 standard deviations from the mean — less than 0.3% probability under a normal distribution. But this method assumes normality, which is often violated.

Handling Outliers

Always investigate before removing. An outlier might be:

1. **A data error:** A salary of $1,000,000 when all others are $30,000–$80,000 — likely a data entry mistake. Fix or remove.
2. **A genuine extreme:** A single order worth $50,000 at an e-commerce site — unusual but real. Removing it would distort the analysis.
3. **A different population:** A commercial account mixed with residential customers — might need to be analyzed separately.

After investigation:

- **Remove** if it is clearly erroneous
- **Cap/winsorize** if you want to limit extreme influence without removing (replace values beyond the bounds with the boundary value)
- **Keep with documentation** if the outlier is real and analytically meaningful

---

4. Correlation and Relationships

Pearson Correlation

Pearson correlation measures the *linear* relationship between two continuous variables. The coefficient *r* ranges from -1 to +1:

- *r* = +1: perfect positive linear relationship
- *r* = 0: no linear relationship
- *r* = -1: perfect negative linear relationship

```
# Single pair
r = df['study_hours_weekly'].corr(df['gpa'])
print(f"Pearson correlation: {r:.3f}")

# Full correlation matrix
corr_matrix = df.select_dtypes(include='number').corr()
```

Interpretation guidelines:	r	< 0.3 is weak, 0.3–0.7 is moderate, > 0.7 is strong. But these are rough — context matters.

Critical limitation: Pearson correlation measures *linear* relationships only. A strong non-linear relationship (U-shape, exponential, quadratic) can produce a Pearson correlation near zero. **Always scatter-plot first.**

Spearman Correlation

Spearman correlation measures *monotonic* relationships — not necessarily linear, but consistently increasing or decreasing. It works on ranks rather than raw values.

```
from scipy import stats

rho, p_value = stats.spearmanr(df['study_hours_weekly'], df['gpa'])
print(f"Spearman rho: {rho:.3f}, p-value: {p_value:.4f}")
```

Use Spearman when:

- The relationship may be non-linear but monotonic
- The data has ordinal variables (rankings, Likert scales)
- Outliers are present (rank-based methods are more robust)

Correlation Matrices and Heatmaps

A correlation matrix shows all pairwise correlations. Visualize it as a heatmap:

```
corr_matrix = df.select_dtypes(include='number').corr()

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, vmin=-1, vmax=1, ax=ax)
ax.set_title('Correlation Matrix — Numeric Variables')
plt.tight_layout()
fig.savefig('output/correlation_heatmap.png', dpi=150)
plt.close()
```

Reading the heatmap: look for dark red (strong positive) and dark blue (strong negative) cells. Near-white cells are near zero. The diagonal is always 1.0 (each variable perfectly correlates with itself).

Edge case — low-variance columns: A column with only one unique value has zero variance. `df.corr()` returns `NaN` for its correlations. Drop such columns before computing the matrix.

Scatter Plots

Before trusting any correlation number, visualize it:

```
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(df['study_hours_weekly'], df['gpa'], alpha=0.5, s=20)
ax.set_xlabel('Study Hours per Week')
ax.set_ylabel('GPA')
ax.set_title(f'Study Hours vs. GPA (r = {r:.2f})')
plt.tight_layout()
fig.savefig('output/scatter_study_gpa.png', dpi=150)
plt.close()
```

Seaborn's `scatterplot` adds grouping by color:

```
fig, ax = plt.subplots(figsize=(8, 6))
sns.scatterplot(x='study_hours_weekly', y='gpa', hue='department',
                data=df, alpha=0.6, ax=ax)
ax.set_title('Study Hours vs. GPA by Department')
plt.tight_layout()
fig.savefig('output/scatter_by_dept.png', dpi=150)
plt.close()
```

Pair Plots

A **pair plot** creates a grid of scatter plots for all pairwise combinations of numeric variables, with distribution histograms on the diagonal:

```
# Best for data sets with ≤10 numeric variables
g = sns.pairplot(df[['gpa', 'study_hours_weekly', 'attendance_pct', 'course_load'
                 diag_kind='kde')
g.fig.suptitle('Pairwise Relationships', y=1.02)
g.savefig('output/pairplot.png', dpi=150)
plt.close()
```

When to use pair plots: For an initial overview with a manageable number of variables (10 or fewer). Beyond that, the grid becomes too large and unreadable. Use the correlation heatmap to identify interesting pairs, then scatter-plot those individually.

Correlation Does Not Equal Causation

This is the most important statistical principle in this module. A strong correlation between two variables does *not* mean one causes the other. Three alternative explanations:

1. **Confounding variable:** Both variables are influenced by a third factor. Ice cream sales and drowning deaths are both driven by summer heat.
2. **Reverse causation:** The direction is backwards. Does studying more cause higher GPA, or does being a high-GPA student lead to more study hours (motivation, selection into harder courses)?
3. **Spurious correlation:** Coincidental co-movement. The divorce rate in Maine correlates with per capita margarine consumption. There is no plausible mechanism.

To establish causation, you need experimental design (random assignment, controlled conditions) or careful causal inference methods. In this course, you observe and report correlations — you do not make causal claims.

Categorical Relationships

For relationships involving categorical variables, use **cross-tabulations** and **grouped bar charts**:

```
# Cross-tabulation
ct = pd.crosstab(df['scholarship'], df['department'])
print(ct)

# Grouped bar chart
fig, ax = plt.subplots(figsize=(10, 6))
ct.plot(kind='bar', ax=ax)
ax.set_title('Scholarship Distribution by Department')
ax.set_ylabel('Count')
plt.tight_layout()
fig.savefig('output/scholarship_by_dept.png', dpi=150)
plt.close()
```

---

5. Hypothesis Testing

The Logic of Hypothesis Testing

You see a pattern in the data — say, students with internships have a 0.3-point higher average GPA than students without. Is this a real difference, or could it have appeared by chance in this particular sample?

Hypothesis testing is a formal framework for answering this question. The logic:

1. **Assume the boring explanation is true.** The **null hypothesis (H_0)** states there is no real effect or difference. "There is no difference in GPA between students with and without internships."
2. **Calculate how surprising your data is under that assumption.** If there were truly no difference, how likely is it that you would see a 0.3-point gap (or larger) just from random sampling?
3. **If the data is very surprising, reject the boring explanation.** If the probability is very low, you conclude that the null hypothesis is unlikely to explain what you see.

p-Values

The **p-value** is the probability of observing data at least as extreme as what you got, *assuming the null hypothesis is true*.

What a p-value IS:

- If p = 0.03, there is a 3% chance of seeing a difference this large (or larger) if there truly is no difference.

What a p-value is NOT:

- It is NOT the probability that H_0 is true.
- It is NOT the probability that your result is a fluke.
- A small p-value does NOT mean the effect is large or important.

Significance Level (alpha)

The **significance level** (α) is the threshold you set *before* looking at the data. By convention, α = 0.05 (5%).

- If p < α: **reject H_0** — the data provides sufficient evidence against the null hypothesis.
- If p ≥ α: **fail to reject H_0** — the data does not provide sufficient evidence. (This does NOT mean H_0 is true — only that you cannot rule it out.)

Type I error (false positive): Rejecting H_0 when it is actually true. Probability = α. At α = 0.05, you accept a 5% false positive rate.

Type II error (false negative): Failing to reject H_0 when there is a real effect. Controlled by sample size and effect size.

Effect Size

Statistical significance tells you whether a difference is real. **Effect size** tells you whether the difference is *meaningful*.

With a large enough sample, even a tiny, practically meaningless difference can be statistically significant (p < 0.001). Effect size separates signal from noise.

Cohen's d for comparing two means:

```
import numpy as np

def cohens_d(group1, group2):
    n1, n2 = len(group1), len(group2)
    var1, var2 = group1.var(), group2.var()
    pooled_std = np.sqrt(((n1 - 1) * var1 + (n2 - 1) * var2) / (n1 + n2 - 2))
    return (group1.mean() - group2.mean()) / pooled_std

d = cohens_d(df[df['has_internship'] == 'Yes']['gpa'],
             df[df['has_internship'] == 'No']['gpa'])
print(f"Cohen's d: {d:.3f}")
```

Interpretation:	d	< 0.2 is small, 0.2–0.8 is medium, > 0.8 is large.

Cramér's V for categorical associations:

```
from scipy.stats import chi2_contingency

ct = pd.crosstab(df['scholarship'], df['department'])
chi2, p, dof, expected = chi2_contingency(ct)
n = ct.sum().sum()
k = min(ct.shape) - 1
cramers_v = np.sqrt(chi2 / (n * k))
print(f"Cramér's V: {cramers_v:.3f}")
```

Interpretation: < 0.1 is negligible, 0.1–0.3 is small, 0.3–0.5 is medium, > 0.5 is large.

The t-Test (Continuous, 2 Groups)

Use a t-test when comparing the means of two groups on a continuous variable.

Independent samples t-test (two separate groups):

```
from scipy import stats

internship = df[df['has_internship'] == 'Yes']['gpa']
no_internship = df[df['has_internship'] == 'No']['gpa']

t_stat, p_value = stats.ttest_ind(internship, no_internship)
print(f"t-statistic: {t_stat:.3f}")
print(f"p-value: {p_value:.4f}")
```

Interpreting the result:

t-statistic: the number of standard errors the difference is from zero. Larger	t	means more evidence against H_0.
- p-value: probability of this t-statistic (or more extreme) under H_0.

Paired samples t-test (`stats.ttest_rel()`): For before/after measurements on the same subjects. Not used in this module's assignments but important to know exists.

Assumptions:

1. Each group is approximately normally distributed (check with Shapiro-Wilk: `stats.shapiro(group)`)
2. Groups have approximately equal variances (check with Levene's test: `stats.levene(group1, group2)`)
3. Observations are independent

For small samples (n < 30), normality matters. For large samples, the Central Limit Theorem makes the t-test robust to non-normality.

When assumptions are violated: Use the **Mann-Whitney U test** (non-parametric alternative): `stats.mannwhitneyu(group1, group2)`. It compares ranks instead of means and does not assume normality.

The Chi-Square Test (Categorical)

Use chi-square when testing the association between two categorical variables.

```
from scipy.stats import chi2_contingency

ct = pd.crosstab(df['scholarship'], df['department'])
chi2, p_value, dof, expected = chi2_contingency(ct)

print(f"Chi-square statistic: {chi2:.3f}")
print(f"p-value: {p_value:.4f}")
print(f"Degrees of freedom: {dof}")
```

Interpreting the result:

- chi^2 statistic: measures how far observed counts deviate from expected counts (what you would see if there were no association).
- p-value: probability of this chi^2 value (or more extreme) if the two variables were truly independent.
- Expected counts: what each cell would contain under independence. If any expected cell count is below 5, the test may not be reliable — consider combining categories.

ANOVA (Continuous, 3+ Groups)

Use ANOVA (Analysis of Variance) when comparing means across three or more groups.

```
from scipy.stats import f_oneway

bio = df[df['department'] == 'Biology']['gpa']
bus = df[df['department'] == 'Business']['gpa']
cs = df[df['department'] == 'Computer Science']['gpa']
eng = df[df['department'] == 'Engineering']['gpa']
math = df[df['department'] == 'Mathematics']['gpa']

f_stat, p_value = f_oneway(bio, bus, cs, eng, math)
print(f"F-statistic: {f_stat:.3f}")
print(f"p-value: {p_value:.4f}")
```

ANOVA tells you *whether* at least one group differs. It does not tell you *which* group. If significant, follow up with pairwise t-tests (and apply Bonferroni correction — see below).

Non-parametric alternative: Kruskal-Wallis test (`stats.kruskal()`) when normality is violated.

statsmodels for Richer Output

`scipy.stats` gives you the basics. `statsmodels` adds confidence intervals, proportions tests, and more detailed output:

```
from statsmodels.stats.weightstats import ttest_ind

t_stat, p_value, df_val = ttest_ind(internship, no_internship)
```

For proportions (e.g., "Is the retention rate different between two groups?"):

```
from statsmodels.stats.proportion import proportions_ztest

count = [success_a, success_b]
nobs = [total_a, total_b]
z_stat, p_value = proportions_ztest(count, nobs)
```

Multiple Comparisons: The Bonferroni Correction

If you run 20 independent hypothesis tests at α = 0.05, you expect 1 false positive even if all null hypotheses are true. The more tests you run, the more likely you are to find a "significant" result by chance.

The **Bonferroni correction** adjusts α by dividing by the number of tests: if you run 10 pairwise comparisons, use α = 0.05/10 = 0.005 for each individual test.

This is conservative — it reduces false positives at the cost of missing real effects. For exploratory work, note which results would survive correction and which would not.

---

6. KPI Design and Cohort Analysis

What Makes a Good KPI

A **Key Performance Indicator (KPI)** is a metric directly tied to a business objective that drives decisions. Not all metrics are KPIs.

- **Revenue per month** is a KPI if the objective is "grow revenue 15% year-over-year."
- **Average page load time** is a metric, not a KPI, unless performance is a strategic priority.

Good KPIs are:

1. **Measurable:** You can compute it from available data.
2. **Actionable:** A change in the KPI triggers a specific business response.
3. **Time-bound:** It has a measurement period (daily, weekly, monthly).
4. **Tied to an objective:** It connects to a business goal the audience cares about.

KPI Framework

For each KPI, document:

Element	Description
Name	Clear, specific (e.g., "Monthly Active Customer Count")
Definition	Precisely what is being measured and how
Formula	The computation (e.g., count of distinct customers with ≥1 order per month)
Data source	Which tables/columns are used
Baseline value	Current value computed from the data
Target	Where it should be (if known) or how to interpret changes
Interpretation	What this number means in business terms

Cohort Analysis

A **cohort** is a group of entities sharing a common characteristic, tracked over time. Cohort analysis reveals trends that aggregate metrics hide.

Examples:

- Customers grouped by registration month: do January signups retain better than March signups?
- Students grouped by department: does GPA trend differ by department?
- Orders grouped by product category: how does revenue composition change over time?

Time-Based Group-Bys

Converting timestamps to periods for aggregation:

```
# Group by month
monthly = df.groupby(df['order_date'].dt.to_period('M')).agg(
    total_revenue=('revenue', 'sum'),
    order_count=('order_id', 'nunique'),
    avg_order_value=('revenue', 'mean')
)

# Resample (requires datetime index)
df.set_index('order_date', inplace=True)
weekly = df.resample('W').agg({
    'revenue': 'sum',
    'order_id': 'nunique'
})
```

Cumulative Metrics

Some KPIs are cumulative:

```
# Running total of revenue
df['cumulative_revenue'] = df.sort_values('order_date')['revenue'].cumsum()

# Retention rate: proportion of Month 1 customers who also ordered in Month N
```

Connecting to Module 3

In Module 3's lab, you wrote a short KPI brief — 3–5 metrics with current values and one-sentence interpretations. Module 4 elevates this to a full KPI *framework*: formal definitions, formulas, baselines, cohort segmentation, and statistical validation. The integration task requires statistical tests to back your KPIs — not just "revenue differs by city" but "the revenue difference between Amman and Irbid is statistically significant (t-test, p = 0.02, Cohen's d = 0.45)."

---

7. Visualization as Communication

Chart Type Selection

Choosing the right chart type is an analytical decision, not an aesthetic one. The question you are answering determines the chart:

Analytical Question	Chart Type	Example
Comparison across categories	Bar chart (vertical or horizontal)	Revenue by product category
Comparison across categories × groups	Grouped bar chart	Revenue by category, split by city
Trend over time	Line chart	Monthly revenue
Trend with cumulative context	Area chart	Cumulative signups
Distribution of a continuous variable	Histogram, density plot	GPA distribution
Distribution comparison across groups	Box plot, violin plot	GPA by department
Relationship between two continuous variables	Scatter plot	Study hours vs. GPA
Relationship with a third variable	Bubble chart, colored scatter	Scatter with size or color encoding
Matrix of values	Heatmap	Correlation matrix, missing data
Composition	Stacked bar	Revenue share by category
Composition (few parts)	Pie chart (use sparingly)	Market share (3–5 segments only)

The pie chart warning: Humans are poor at comparing angles and areas. Pie charts work only with 3–5 segments of notably different sizes. For anything else, use a bar chart.

Design Principles

Gestalt principles explain how humans perceive visual groups:

- **Proximity:** Items close together are perceived as related. Group related chart elements spatially.
- **Similarity:** Items that look alike (same color, shape) are perceived as belonging together. Use consistent color for the same category across charts.
- **Enclosure:** Items within a boundary are perceived as a group. Use panels (subplots) to group related charts.
- **Connection:** Items connected by lines are perceived as related. Use lines for sequential data (time series).

Every chart needs a title that states the finding, not just the data. Compare:

- Bad title: "Revenue by Month"
- Good title: "Revenue Peaked in September (+40% over August)"

The title is the first thing anyone reads. It should tell them what to see, not what to look for.

Color

Use colorblind-safe palettes. Approximately 8% of men have color vision deficiency — in a room of 50 learners, expect several.

- **Sequential palettes** (light to dark): for continuous values. Matplotlib: `viridis`, `plasma`. Seaborn: `rocket`, `mako`.
- **Diverging palettes** (two colors with white center): for values centered around zero. Matplotlib: `coolwarm`, `RdBu`. Use `center=0` in Seaborn heatmaps.
- **Qualitative palettes** (distinct colors): for categorical variables. Seaborn: `colorblind`, `Set2`.

```
# Set a global colorblind-safe palette
sns.set_palette('colorblind')
```

Matplotlib Deep Dive

Matplotlib is the foundational plotting library in Python. Seaborn, Plotly, and most other visualization libraries build on or interoperate with it. Understanding Matplotlib's model gives you full control.

The figure-axes model:

```
# Create a figure (the canvas) and axes (the plot area)
fig, ax = plt.subplots(figsize=(8, 5))

# Plot on the axes
ax.bar(categories, values, color='steelblue')

# Customize the axes
ax.set_xlabel('Category', fontsize=12)
ax.set_ylabel('Revenue (JOD)', fontsize=12)
ax.set_title('Revenue by Category', fontsize=14)

# Add annotation
ax.annotate('Highest', xy=(2, 45000), xytext=(3, 48000),
            arrowprops=dict(arrowstyle='->', color='red'),
            fontsize=10, color='red')

# Layout and save
plt.tight_layout()
fig.savefig('output/revenue_by_category.png', dpi=150, bbox_inches='tight')
plt.close()
```

Multi-panel figures:

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Top left
axes[0, 0].hist(df['gpa'], bins=20, edgecolor='black')
axes[0, 0].set_title('GPA Distribution')

# Top right
sns.boxplot(x='department', y='gpa', data=df, ax=axes[0, 1])
axes[0, 1].set_title('GPA by Department')
axes[0, 1].tick_params(axis='x', rotation=45)

# Bottom left
axes[1, 0].scatter(df['study_hours_weekly'], df['gpa'], alpha=0.5)
axes[1, 0].set_title('Study Hours vs. GPA')

# Bottom right
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
            ax=axes[1, 1])
axes[1, 1].set_title('Correlation Matrix')

plt.tight_layout()
fig.savefig('output/eda_summary.png', dpi=150)
plt.close()
```

Key Matplotlib patterns:

Task	Code
Create figure + axes	`fig, ax = plt.subplots(figsize=(w, h))`
Multiple panels	`fig, axes = plt.subplots(rows, cols)`
Set labels	`ax.set_xlabel()`, `ax.set_ylabel()`
Set title	`ax.set_title()`
Add annotation	`ax.annotate(text, xy=..., xytext=...)`
Rotate tick labels	`ax.tick_params(axis='x', rotation=45)`
Add legend	`ax.legend()`
Tight layout	`plt.tight_layout()`
Save to file	`fig.savefig('path.png', dpi=150)`
Close figure	`plt.close()`

Jupyter vs. script behavior:

- In Jupyter notebooks: `%matplotlib inline` renders charts in the notebook. `plt.show()` displays immediately.
- In scripts: `plt.show()` opens a window and blocks execution until closed. For scripts, save with `fig.savefig()` and close with `plt.close()`. Do not call `plt.show()` in scripts that run without a display (like CI environments).

Seaborn Integration

Seaborn wraps Matplotlib with sensible defaults for statistical visualization. It produces Matplotlib objects — you can customize them with Matplotlib methods.

```
# Set a global theme
sns.set_theme(style='whitegrid', palette='colorblind')
```

Axes-level functions (return a Matplotlib Axes, plot on an existing axis):

- `sns.histplot()`, `sns.kdeplot()`, `sns.boxplot()`, `sns.violinplot()`, `sns.scatterplot()`, `sns.heatmap()`

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.histplot(df['gpa'], kde=True, ax=ax)
ax.set_title('Custom title using Matplotlib')
```

Figure-level functions (create their own Figure, cannot be placed on existing axes):

- `sns.displot()`, `sns.catplot()`, `sns.relplot()`, `sns.pairplot()`

```
g = sns.displot(df, x='gpa', hue='department', kind='kde')
g.fig.suptitle('KDE by Department', y=1.02)
```

Do not mix these. A figure-level function creates its own figure. Passing `ax=` to a figure-level function does not work. Use axes-level functions when building custom multi-panel layouts with `plt.subplots()`.

When to use Seaborn over raw Matplotlib:

- Statistical patterns (distributions, correlations, group comparisons) — Seaborn handles aesthetics and data grouping automatically
- EDA — Seaborn's defaults produce clean, readable charts with minimal code
- Heatmaps — `sns.heatmap()` is significantly easier than building one in raw Matplotlib

When to use raw Matplotlib:

- Full control over every element (custom annotations, unusual layouts)
- Non-statistical charts (custom diagrams, specialized formatting)
- When Seaborn's abstractions get in the way

Plotly (Conceptual Overview)

Plotly creates **interactive** visualizations — hover tooltips, zoom, pan, and filter. It is installed from Extended Integration Day 1 (EID 1) and covered here conceptually. **Plotly is not required in any graded deliverable** — it is available for exploration and challenge tiers.

When interactivity matters: Dashboards for non-technical audiences, exploratory tools where users need to drill into data, standalone HTML reports that stakeholders can interact with.

Plotly Express provides a high-level API similar to Seaborn:

```
import plotly.express as px

fig = px.bar(df_monthly, x='month', y='revenue', color='category',
             title='Monthly Revenue by Category')
fig.update_layout(
    xaxis_title='Month',
    yaxis_title='Revenue (JOD)',
    legend_title='Category'
)
fig.write_html('output/revenue_interactive.html')
```

```
fig = px.scatter(df, x='study_hours_weekly', y='gpa',
                 color='department', size='attendance_pct',
                 title='Study Hours vs GPA by Department')
fig.write_html('output/scatter_interactive.html')
```

Key Plotly patterns:

Task	Code
Bar chart	`px.bar(df, x=..., y=..., color=...)`
Line chart	`px.line(df, x=..., y=..., color=...)`
Scatter plot	`px.scatter(df, x=..., y=..., color=...)`
Update layout	`fig.update_layout(title=..., xaxis_title=...)`
Save as HTML	`fig.write_html('output/chart.html')`
Show in notebook	`fig.show()`

Environment notes:

- `fig.show()` opens a browser tab in scripts; renders inline in Jupyter.
- `fig.write_html()` produces a standalone HTML file — no server needed. This is the recommended output for reports.
- `fig.write_image()` requires the `kaleido` package for static image export. Not installed by default.

When to use which library:

Library	Strength	Use When
Matplotlib	Full control, static publication quality	You need precise layout, annotation, or non-standard charts
Seaborn	Statistical patterns, EDA defaults	You are exploring distributions, correlations, or group comparisons
Plotly	Interactivity, hover, zoom, standalone HTML	Your audience needs to explore the data themselves

In this course: Matplotlib and Seaborn for all graded work. Plotly for challenge tiers and independent exploration.

Troubleshooting

`sns.histplot()` raises an error about KDE If a column has very few unique values or all identical values, the KDE estimation can fail. Add `kde=False` or ensure the column has sufficient variance.

Matplotlib figure is blank after `plt.savefig()` You may have called `plt.show()` before `plt.savefig()`. `plt.show()` clears the figure. Always save before showing, or use the `fig.savefig()` object method.

Seaborn figure-level function ignores `ax` parameter Figure-level functions (`displot`, `catplot`, `relplot`, `pairplot`) create their own figures. Pass `ax=` only to axes-level functions (`histplot`, `boxplot`, `heatmap`, etc.).

`df.corr()` returns NaN for a column The column likely has zero variance (one unique value) or is entirely null. Drop it before computing correlations.

t-test returns NaN One or both groups may have zero variance (all identical values) or be empty. Check group sizes: `len(group_a)`, `len(group_b)`.

Matplotlib in Jupyter vs. script In Jupyter: `%matplotlib inline` at the top of the notebook. In scripts: use `fig.savefig()` and `plt.close()`. Do not rely on `plt.show()` in scripts that run unattended.

Seaborn `heatmap` text overlaps Use `fmt='.1f'` instead of `.2f` to reduce decimal places, or increase `figsize`. For large matrices, set `annot=False` and rely on the color scale.

p-value is exactly 0.0 Very rare but possible with large samples and strong effects. Report as "p < 0.001" rather than "p = 0.0" — the actual value is not zero, just below floating-point resolution.

Empty groups in `groupby()` If a cohort has zero records, `groupby().mean()` returns NaN for that group. Handle with `.fillna(0)` if zero is appropriate, or document the empty cohort.

Categorical vs. numeric confusion A column like `customer_tier` with values 1, 2, 3 may be `int64` but is semantically categorical. Convert before analysis: `df['customer_tier'] = df['customer_tier'].astype('category')`. This affects how Seaborn handles the variable in plots.

Skewed data and mean imputation Imputing missing values with the mean when the distribution is heavily skewed inflates the central tendency and reduces variance. Use the median for skewed distributions.

Small sample sizes in hypothesis tests t-tests assume normality for small samples (n < 30). Check with Shapiro-Wilk: `stat, p = stats.shapiro(group)`. If p < 0.05, the data is not normal — use the Mann-Whitney U test instead.

Key Terms

Term	Definition
box plot / box-and-whisker plot	A chart showing median, quartiles, and outliers of a distribution
chi-square test	A statistical test for association between categorical variables
cohort	A group of entities sharing a common characteristic, tracked over time
confidence interval	A range of values within which the true parameter is expected to fall with a given probability
correlation coefficient	A measure (−1 to +1) of strength and direction of a linear (Pearson) or monotonic (Spearman) relationship
descriptive analytics	Summarizing and interpreting historical data to understand what happened and why
distribution	The pattern of values a variable takes, including shape, center, and spread
effect size	A quantitative measure of the magnitude of a phenomenon, independent of sample size
EDA (Exploratory Data Analysis)	Systematic process of examining a dataset's structure, distributions, relationships, and anomalies before formal modeling
heatmap	A matrix visualization using color intensity to represent values
histogram	A chart showing frequency distribution of a continuous variable, divided into bins
hypothesis testing	A statistical method for deciding whether observed data provides sufficient evidence against a null hypothesis
imputation	Replacing missing values with estimated values (mean, median, mode, or model-based)
interquartile range (IQR)	Q3 − Q1; the range of the middle 50% of values
KDE (Kernel Density Estimation)	A smooth, continuous estimate of a variable's probability density function
KPI (Key Performance Indicator)	A metric directly tied to a business objective that drives decisions
MCAR / MAR / MNAR	Classifications of missing data mechanisms: Missing Completely At Random, Missing At Random, Missing Not At Random
null hypothesis (H_0)	The default assumption of no effect or no difference
outlier	A data point far outside the expected range of values
p-value	The probability of observing data at least as extreme as the current data, assuming H_0 is true
pair plot	A grid of scatter plots showing pairwise relationships between all variables
significance level (α)	The threshold (typically 0.05) below which the p-value leads to rejecting H_0
skewness	A measure of asymmetry of a distribution; positive = right tail, negative = left tail
t-test	A statistical test comparing the means of two groups
violin plot	A combination of box plot and density plot showing distribution shape alongside summary statistics

---

Summary

- EDA is the first step before any analysis: assess shape, types, distributions, relationships, and anomalies
- Use median for skewed data, mean for symmetric; always check the distribution shape before choosing
- Missing data is not just NaN — watch for sentinel values and implicit gaps; handle based on the missingness pattern (MCAR/MAR/MNAR)
- Correlation measures association, not causation; always scatter-plot before trusting a coefficient; Pearson is linear only
- Hypothesis testing answers "can chance explain this pattern?"; p-values are probabilities under H_0, not probabilities that H_0 is true
- Effect size separates statistical significance from practical significance — both matter
- KPIs are metrics tied to business objectives; a good KPI framework includes definition, formula, baseline, target, and interpretation
- Chart type is an analytical choice: comparison (bar), trend (line), distribution (histogram/box), relationship (scatter), matrix (heatmap)
- Matplotlib gives you full control; Seaborn gives you statistical defaults; Plotly gives you interactivity
- Every chart title should state the finding, not just describe the data

---

What's Next

Tonight (Day 1): The drill — three functions: summary statistics, distribution plots, and a correlation heatmap. Write them from memory.

Monday (Day 2): The lab — a full EDA on university student performance data. Distributions, correlations, hypothesis tests, and a findings report.

Wednesday (Day 4): The integration — a KPI dashboard for the Amman Digital Market. Extract from Postgres, define KPIs, validate with statistics, and produce an executive summary.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.